

一、文件和目录操作

创建文件和目录

- `touch [文件名]`
 - 用途：创建一个空文件或者更新现有文件的时间戳。
 - 示例：创建一个名为`example.txt`的空文件。

```
touch example.txt
```

- `mkdir [目录名]`
 - 用途：创建一个新的目录。
 - 示例：创建一个名为`new_folder`的目录。

```
mkdir new_folder
```

删除文件和目录

- `rmdir [目录名]`
 - 用途：删除一个空的目录。如果目录中有文件或其他目录，`rmdir`会失败。
 - 示例：删除一个名为`empty_folder`的空目录。

```
rmdir empty_folder
```

- `rm [选项] [文件或目录]`
 - 用途：删除文件或目录。使用`-r`选项可以递归删除目录及其内容。
 - 示例：删除名为`old_folder`的目录及其所有内容。

```
rm -r old_folder
```

实用技巧

- 使用通配符（如`*`）来操作多个文件。例如，删除所有`.txt`文件：

```
rm *.txt
```

二、文件查看和处理

查看文件内容

- **cat** [文件名]

- 用途：显示整个文件的内容。
- 示例：查看file.txt的内容。

```
cat file.txt
```

- **less** [文件名]

- 用途：分页查看文件内容，适合查看较大的文件。
- 示例：分页查看large_file.txt。

```
less large_file.txt
```

- **head** [文件名]

- 用途：查看文件的前几行，默认为前10行。
- 示例：查看example.txt的前5行。

```
head -n 5 example.txt
```

- **tail** [文件名]

- 用途：查看文件的最后几行，默认为最后10行。
- 示例：实时监控log.txt的最新内容。

```
tail -f log.txt
```

文本搜索

- **grep** [选项] [模式] [文件名]

- 用途：搜索文件中包含特定模式的行。模式可以是字符串或正则表达式。
- 示例：在document.txt中搜索包含"example"的行。

```
grep "example" document.txt
```

文件排序

- **sort [文件名]**
 - 用途：对文件的行进行排序。
 - 示例：对names.txt的内容按字典顺序排序。

```
sort names.txt
```

文件比较

- **diff [文件1] [文件2]**
 - 用途：比较两个文件的内容差异。
 - 示例：比较version1.txt和version2.txt的差异。

```
diff version1.txt version2.txt
```

三、权限和所有权

查看文件权限

- **ls -l [文件名]**
 - 用途：列出文件或目录的详细信息，包括权限、所有者和组。
 - 示例：查看example.txt的详细信息。

```
ls -l example.txt
```

修改文件权限

- **chmod [权限] [文件名]**
 - 用途：更改文件或目录的权限。权限可以用数字（如755）或符号（如u+x）表示。
 - 示例：给script.sh文件添加执行权限。

```
chmod +x script.sh
```

修改文件所有权

使用sudo执行命令

- **sudo [命令]**
 - 用途：以超级用户（通常是root）的身份执行命令。
 - 示例：使用root权限安装软件包。

```
sudo apt-get install [包名]
```

- `chown` [所有者][:组] [文件名]

- 用途：更改文件或目录的所有者和/或组。
- 示例：将`data.txt`的所有者更改为`user`，组更改为`group`。

```
chown user:group data.txt
```

权限解释

- 文件权限由三组字符表示，分别对应所有者（owner）、组（group）和其他用户（others）的权限。
- 每组权限由三个字符组成，分别表示读（r）、写（w）和执行（x）权限。
- 例如，权限`rwxr-xr--`表示所有者拥有读写执行权限，组拥有读和执行权限，其他用户只有读权限。

四、系统信息和进程管理

系统信息查询

- `uname -a`

- 用途：显示系统信息，包括内核版本和硬件架构等。
- 示例：显示全部系统信息。

```
uname -a
```

- `top`

- 用途：实时显示系统进程信息和资源使用情况。
- 示例：直接运行`top`命令以查看信息。

```
top
```

- `df -h`

- 用途：显示磁盘空间使用情况，`-h`参数使输出易于阅读。
- 示例：查看磁盘使用情况。

```
df -h
```

- `free -m`

- 用途：显示内存使用情况，`-m`参数表示以MB为单位显示。
- 示例：查看内存使用情况。

```
free -m
```

进程管理

- `ps aux`

- 用途：显示当前系统的所有进程信息。
- 示例：查看所有进程。

```
ps aux
```

- `kill [进程ID]`

- 用途：终止指定ID的进程。
- 示例：终止进程ID为1234的进程。

```
kill 1234
```

- `screen`

- 用途：允许你在单个终端窗口中使用多个shell会话。
- 示例：启动新的screen会话。

```
screen
```

五、网络工具

检查网络连接

- `ping [地址]`

- 用途：发送ICMP ECHO_REQUEST包到网络主机以测试连接。
- 示例：测试与google.com的连接。

```
ping google.com
```

查看网络配置

- `ifconfig`

- 用途：显示或配置网络接口的网络配置。
- 示例：查看所有网络接口的配置。

```
ifconfig
```

端口和连接信息

- **netstat -tuln**
 - 用途：显示网络连接、路由表、接口统计等网络信息，**-tuln**参数表示仅显示TCP/UDP的监听端口。
 - 示例：查看监听端口。

```
netstat -tuln
```

DNS解析

- **dig [域名]**
 - 用途：查询DNS信息，获取域名的地址记录等。
 - 示例：查询example.com的DNS信息。

```
dig example.com
```

六、管道与重定向

管道 (|)

- 用途：将一个命令的输出作为另一个命令的输入。
- 示例：使用**grep**搜索**ps**命令的输出。

```
ps aux | grep apache
```

输出重定向(>和>>)

- **>**
 - 用途：将命令的输出写入到文件中，如果文件已存在，则覆盖。
 - 示例：将ls命令的输出保存到filelist.txt。

```
ls > filelist.txt
```

- **>>**

- 用途：将命令的输出追加到文件中，而不是覆盖文件。
- 示例：追加输出到`filelist.txt`。

```
ls >> filelist.txt
```

七、Make指南

1. 简介

Make工具通过读取`Makefile`来自动化构建过程，减少重复的编译指令输入。`Makefile`定义了一系列的规则来指定如何生成目标文件。

2. 基本语法

一个基本的`Makefile`包含以下几个部分：

- 目标（**Targets**）：指明要构建的文件。
- 依赖（**Dependencies**）：目标文件依赖的文件，只有当依赖文件比目标文件新，或目标文件不存在时，才会执行相应的命令。
- 命令（**Commands**）：构建目标文件的具体命令，必须以Tab键开始。

3. 示例Makefile

```
# 示例Makefile
all: program

program: main.o lib.o
    gcc -o program main.o lib.o

main.o: main.c
    gcc -c main.c

lib.o: lib.c
    gcc -c lib.c

clean:
    rm -f program *.o
```

- `all`是一个约定俗成的目标，用来构建项目中的所有目标文件。
- `program`依赖于`main.o`和`lib.o`，如果这些`.o`文件比`program`更新，`make`会重新构建`program`。
- `main.o`和`lib.o`分别依赖于其对应的`.c`源文件。
- `clean`是一个伪目标，用于删除所有编译生成的文件，以便从头开始重新构建。

4. 使用Make

- 构建项目:在`Makefile`所在目录下，运行`make`命令。如果没有指定目标，默认构建第一个目标（在上面的示例中是`all`）。

- 清理构建:运行`make clean`来删除所有编译产生的文件。

5.常用函数

Makefile还支持一些函数，用于文本处理、文件操作等。例如，使用`$(wildcard *.c)`来获取所有.c文件的列表。

6.自动变量

- `$@` 表示当前规则中的目标文件。
- `^` 表示当前规则中所有的依赖文件。
- `$<` 表示当前规则中的第一个依赖文件。

7.并行构建

使用`make -j [number]`命令可以并行构建项目，其中`[number]`是并行任务的数目。这可以显著加速构建过程。